MANUAL TÉCNICO

Arquitectura y desarrollo

Cómo está construido MedCloud, qué tecnologías usa, dónde vive la lógica de negocio, cómo instalarlo y cómo extenderlo.

PRODUCTO	VERSIÓN	FECHA
MedCloud · Gestión clínica integral	React 19 · Node · SQL Server	Octubre 2026

Contenido

1	Visión general INTRODUCCIÓN	2
2	Arquitectura ARQUITECTURA	3
3	Instalación INSTALACIÓN	5
4	Base de datos BASE DE DATOS	7
5	API API	9
6	Módulos MÓDULOS	13
A	Historial de cambios HISTORIAL	16

Datos de salud y base productiva

MedCloud guarda datos de salud (Ley 25.326 y Ley 26.529). La base del servidor local es productiva: no se prueba contra ella y al levantar el backend se envían mails reales. Este documento no incluye contraseñas ni claves: solo el nombre de cada variable y dónde se configura.

1 INTRODUCCIÓN

Visión general

MedCloud es un sistema web de gestión para clínicas y consultorios: pacientes, agenda de turnos, historia clínica, caja diaria, cobros y liquidación de honorarios médicos. Lo usan tres roles: **ADMIN**, **RECEPCION** y **MEDICO**.

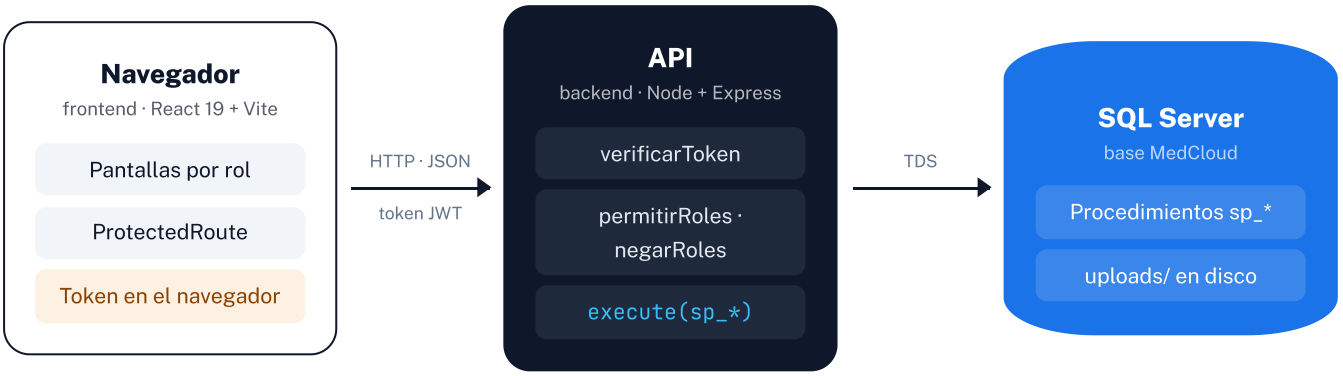


Figura 1. Capas del sistema. La API valida sesión y rol en cada ruta, ejecuta el procedimiento almacenado y responde.

El sistema tiene tres capas: un frontend en React (Vite) que corre en el navegador, una API en Node + Express y una base SQL Server donde vive la mayor parte de la lógica en procedimientos almacenados (`sp_*`). La API valida la sesión (JWT) y el rol en cada ruta antes de llamar al procedimiento.

2 ARQUITECTURA

Arquitectura

MedCloud separa navegador, API y base de datos. La API es delgada: recibe el pedido, valida sesión y rol, ejecuta un procedimiento almacenado y responde.

Carpetas

```
MedCloud-Backend/
  src/index.js              arranque: rutas, archivos de /uploads, scheduler de recordatorios
  src/config.js             configuración sensible del .env (JWT_SECRET); si falta, no arranca
  src/db.js                 pool de conexión a SQL Server
  src/middlewares/          auth.middleware.js: verificarToken, permitirRoles, negarRoles
  src/routes/               definición de endpoints y permisos por ruta
  src/controllers/          llaman a los procedimientos almacenados
  src/utils/                emailSender, scheduler, reciboHelper, phoneFormatter, roles
  uploads/                  estudios, comprobantes y recibos (no se versiona)
frontend/
  src/main.jsx              axios global: agrega el token y la URL del backend (API_URL)
  src/App.jsx               rutas, ProtectedRoute por rol, sesión vencida, inactividad, trial
  src/config.js             API_URL
  src/utils/                auth.js (usuario y rol), archivos.js (abrir archivos con sesión)
  src/components/           una pantalla por componente
database/DER.md             modelo de datos documentado
docs/                       manuales de usuario y técnico
```

Sesión

1. `POST /login` ejecuta `sp_LoginUsuario`, compara la contraseña y firma un JWT (8 h) con `id`, `username`, `rol`, `trial_dias_restantes` y un `session_id` aleatorio que también se guarda en `usuarios.session_token`.
2. El frontend guarda el token en `localStorage` y el interceptor de `main.jsx` lo envía como `Authorization: Bearer <token>` en cada pedido.
3. `verificarToken` valida firma y vencimiento con `JWT_SECRET`, compara `session_id` con el de la base (sesión única: si el usuario entra en otro equipo, la sesión anterior recibe `401 CONCURRENT_LOGIN_DISPLACED`) y bloquea las escrituras si el trial está vencido (`403 TRIAL_EXPIRED`).
4. Cualquier `401` con sesión guardada borra la sesión en el navegador y lleva al ingreso con el aviso "Sesión vencida" (o "Sesión finalizada" si fue desplazada). También se cierra a los 15 minutos sin actividad.

Permisos

- `permitirRoles(...)` deja pasar solo a los roles indicados; `negarRoles(...)` bloquea a los indicados. Se usan después de `verificarToken`.
- Regla del proyecto: **el backend replica lo que muestra el frontend**. Si una pantalla es solo para ADMIN y RECEPCION (`ProtectedRoute allowedRoles`), su API usa `permitirRoles('ADMIN', 'RECEPCION')`; si un botón se oculta al MEDICO, su API usa `negarRoles('MEDICO')`. La matriz completa está en api.md.

- Escalada de privilegios: aunque RECEPCION administra usuarios y roles, solo un ADMIN puede crear, editar o borrar usuarios con rol ADMIN, asignar ese rol o tocar el rol ADMIN (`src/utils/roles.js`).
- Historia clínica: solo MEDICO, y el médico se toma siempre del token (`req.user.id`), nunca de la URL o del cuerpo.

Archivos subidos

Los archivos de `uploads/` (estudios, comprobantes de transferencia, recibos de efectivo generados en HTML) se sirven solo con sesión en `GET /uploads/:archivo` . El nombre se reduce con `path.basename` para que no se pueda salir de la carpeta. Si un recibo de efectivo no existe en disco, se regenera a partir del pago. En el frontend, `utils/archivos.js` los descarga con el token y los abre en una pestaña nueva (la pestaña se abre en el mismo clic para que el navegador no la bloquee).

Decisiones de diseño

- **Sin edición ni borrado de evoluciones clínicas**: la historia clínica es inviolable (Ley 26.529, art. 18). Las rutas `PUT/DELETE /historial/:id` se quitaron; ninguna pantalla las usaba.
- **Falla temprana de configuración**: sin `JWT_SECRET` (mínimo 32 caracteres) el servidor no arranca, para que nunca funcione con una firma conocida.
- **Recordatorios**: `utils/scheduler.js` corre dentro del mismo proceso cada 10 minutos (pacientes del día siguiente y agenda de cada médico a las 20 h). Si se levantan dos instancias, se duplican los mails.

3 INSTALACIÓN

Instalación

Requisitos: Node 20 o superior, SQL Server con la base `MedCloud` y sus procedimientos `sp_*`, y una cuenta SMTP para los correos.

Variables de entorno del backend

Se configuran en `MedCloud-Backend/.env` (no se versiona). La plantilla es `MedCloud-Backend/.env.example`.

VARIABLE	OBLIGATORIA	USO
`PORT`	no	Documentada; hoy el servidor escucha fijo en 3000
`DB_HOST`, **`DB_PORT`**, **`DB_NAME`**, **`DB_USER`**, **`DB_PASSWORD`**, **`DB_ENCRYPT`**	sí	Conexión a SQL Server (`src/db.js`)
`JWT_SECRET`	**sí**	Firma de las sesiones. Mínimo 32 caracteres; sin ella el servidor no arranca
`SMTP_HOST`, **`SMTP_PORT`**, **`SMTP_USER`**, **`SMTP_PASS`**	sí	Envío de recordatorios, agendas, soporte y recuperación de contraseña
`FRONTEND_URL`	sí	Base de los links de recuperación de contraseña
`BACKEND_URL`	sí	Base de los links de confirmación de turnos en los mails

Para generar un `JWT_SECRET`:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Atención

Cambiar `JWT_SECRET` cierra todas las sesiones abiertas: los usuarios tienen que volver a ingresar una vez.

Frontend

`frontend/.env.production` define la URL del backend (`REACT_APP_API_URL`; también se acepta `VITE_API_URL`). Si no está, `src/config.js` usa `https://api.medcloud.ar` fuera de localhost y `http://localhost:3000` en desarrollo.

Puesta en marcha (desarrollo)

```
npm --prefix MedCloud-Backend install
npm --prefix MedCloud-Backend run dev
```

```
npm --prefix frontend install
npm --prefix frontend run dev
```

Atención

Al arrancar, el backend inicia el scheduler de recordatorios y manda mails reales a los pacientes de la base configurada. No levantarlo apuntando a la base de producción salvo en el servidor de producción.

Despliegue

1. Agregar `JWT_SECRET` al `.env` del servidor **antes** de actualizar el código (si falta, el backend no arranca).
2. `npm --prefix MedCloud-Backend install` (instala `jsonwebtoken`, ahora declarada como dependencia).
3. `npm --prefix frontend run build` y publicar `frontend/dist`.
4. Reiniciar el backend y verificar el ingreso con un usuario de cada rol.

Frontend y backend deben actualizarse juntos: el frontend nuevo abre los archivos con sesión y usa `GET /historial/paciente/:id`.

4 BASE DE DATOS

Base de datos

SQL Server, base `MedCloud`. El modelo está documentado en `database/DER.md`.

> **Atención**
> Los procedimientos almacenados **no están versionados en el repositorio**: existen solo en la base. Exportarlos a `database/` (un archivo por objeto, `CREATE OR ALTER`, como en StockyApp) está pendiente.

Procedimientos que usa el backend

MÓDULO	PROCEDIMIENTOS
Sesión	`sp_LoginUsuario`, `sp_ChangePassword`, `sp_GetUsuarioByEmail`, `sp_CreatePasswordResetToken`, `sp_ValidatePasswordResetToken`, `sp_ResetPasswordByToken`
Usuarios y roles	`sp_GetUsuarios`, `sp_GetUsuarioById`, `sp_CreateUsuario`, `sp_SetUsuario`, `sp_DeleteUsuario`, `sp_GetRoles`, `sp_CreateRol`, `sp_SetRoles`, `sp_DeleteRol`
Administrativos	`getAdministrativos`, `createAdministrativo`, `setAdministrativo`, `deleteAdministrativo`
Profesionales	`sp_GetProfesionales`, `sp_GetProfesional`, `sp_CreateProfesional`, `sp_SetProfesional`, `sp_DeleteProfesional`, `sp_GetEspecialidades`
Pacientes	`sp_GetPacientes`, `sp_GetPaciente`, `sp_CreatePaciente`, `sp_SetPaciente`, `sp_DeletePaciente`, `sp_GetObrasSociales`, `sp_GetHistorialAsistenciaPaciente`
Historia clínica	`sp_GetHistorialesClinicos`, `sp_CreateHistorialClinico`, `sp_Set_CompartirHistorial`
Archivos	`sp_UploadArchivo`, `sp_GetArchivosPaciente`
Turnos	`sp_GetTurnos`, `sp_CreateTurno`, `sp_SetTurno`, `sp_DeleteTurno`, `sp_GetHorarios`, `sp_RegistrarPagoTurno`, `sp_CambiarEstadoTurno`
Caja y pagos	`sp_AbrirCaja`, `sp_CerrarCaja`, `sp_GetLiquidacionProfesional`, `sp_GetHistorialPagosPaciente`, `sp_GetHistorialPagosProfesional`, `sp_RegistrarPagoProfesional`

También hay consultas SQL armadas en el código (pendientes de pasar a procedimientos): caja activa e historial (`caja.controllers.js`), recordatorios y confirmación pública (`turnos.controllers.js`, `utils/scheduler.js`), regeneración de recibos (`index.js`), control de sesión única (`auth.middleware.js`) y guardado de `session_token` en el login.

Supuestos del código sobre la base

- `sp_GetRoles` devuelve `id` y `codigo`. Los códigos de rol usados son `ADMIN`, `RECEPCION` y `MEDICO`; `src/utils/roles.js` los usa para impedir la escalada de privilegios.
- `sp_GetUsuarioById` devuelve `rol_id`.

- `sp_GetHistorialesClinicos` y `sp_CreateHistorialClinico` reciben `usuario_id` (el usuario del token) y resuelven el profesional y la privacidad dentro del procedimiento.
- `usuarios.session_token` guarda la sesión activa (la columna se crea en el primer login si no existe).
- Estados de turno (`CK_Turnos_Estado`): `Pendiente` , `Confirmado` , `Completado` , `Cancelado` .

Pendiente de verificar con acceso de solo lectura

- Si `sp_GetUsuarios` o `sp_GetUsuarioById` devuelven `password_hash` .
- Si `sp_SetProfesional` conserva horarios y DNI (el backend no los envía al editar).
- Si `sp_Set_CompartirHistorial` valida que quien comparte sea el autor de la evolución.
- Columna de especialidad vigente en `profesionales` (`especialidad` en texto o `especialidad_id`).

5 API

API

Todas las rutas, salvo las públicas indicadas, requieren `Authorization: Bearer <token>`.

Roles: **A** = ADMIN, **R** = RECEPCION, **M** = MEDICO, **Todos** = cualquier usuario con sesión. "Todos menos M" significa que cualquier rol excepto MEDICO puede usarla.

Errores comunes de sesión y permisos:

CÓDIGO	CUÁNDO
`401`	Falta el token, es inválido o venció (`Token inválido o expirado`)
`401` `CONCURRENT_LOGIN_DISPLACED`	El usuario inició sesión en otro equipo
`403`	El rol no tiene permiso (`No tenés permisos para realizar esta acción.`)
`403` `TRIAL_EXPIRED`	Período de prueba vencido: solo se permiten lecturas (GET)
`500`	Error de base de datos (texto del error)

Públicas

MÉTODO	RUTA	USO
POST	`/login`	Ingreso. Cuerpo `{ username, password }` → `{ token, user }`
POST	`/auth/forgot-password`	Pide el mail de recuperación. Responde siempre el mismo mensaje
POST	`/auth/validate-reset-token`	Valida el link de recuperación
POST	`/auth/reset-password`	Cambia la contraseña con el link
GET	`/turnos/accion-publica?id=&accion=`	Confirmar o cancelar desde el mail (en desarrollo, todavía sin firma; ver changelog)

Sesión

MÉTODO	RUTA	ROLES	USO
GET	`/auth/verify-session`	Todos	Control de sesión única (cada 30 s desde el frontend)

Usuarios

MÉTODO	RUTA	ROLES	NOTAS
GET	`/usuarios`, `/usuarios/:id`	A, R	
POST	`/usuarios`	A, R	Solo A puede crear con rol ADMIN (`403`)
PUT	`/usuarios/:id`	A, R	Solo A puede editar a un ADMIN o asignar el rol ADMIN (`403`)
DELETE	`/usuarios/:id`	A, R	Solo A puede borrar a un ADMIN (`403`)
PUT	`/usuarios/:id/password`	Todos	Solo la propia contraseña; A puede cambiar cualquiera (`403 Solo podés cambiar tu propia contraseña.`)

Roles

MÉTODO	RUTA	ROLES	NOTAS
GET	`/roles`	A, R	
POST / PUT / DELETE	`/roles`, `/roles/:id`	A, R	Solo A puede crear, editar o borrar el rol con código ADMIN (`403`)

Profesionales

MÉTODO	RUTA	ROLES
GET	`/profesionales`, `/profesionales/:id`, `/especialidades`	Todos
POST / PUT / DELETE	`/profesionales`, `/profesionales/:id`	A, R

Administrativos

MÉTODO	RUTA	ROLES	NOTAS
GET / POST / PUT / DELETE	`/administrativos`, `/administrativos/:id`	A	Ninguna pantalla las usa

Pacientes

MÉTODO	RUTA	ROLES	NOTAS
GET	`/pacientes`	Todos	`sp_GetPacientes` filtra por usuario (el médico ve los suyos)
GET	`/pacientes/:id`	Todos	

MÉTODO	RUTA	ROLES	NOTAS
GET	`/pacientes/:id/asistencia`	Todos menos M	
POST / PUT	`/pacientes`, `/pacientes/:id`	A, R	
DELETE	`/pacientes/:id`	A, R	Responde `204`
GET	`/obras-sociales`	Todos	

Historia clínica

El médico se identifica siempre por el token, nunca por la URL ni por el cuerpo.

MÉTODO	RUTA	ROLES	NOTAS
GET	`/historial/paciente/:paciente_id`	M	
GET	`/historial/paciente/:paciente_id/profesional/:x`	M	Ruta anterior; `:x` se ignora
POST	`/historial`	M	`{ paciente_id, turno_id, motivo, evolucion, diagnostico, tratamiento }`
POST	`/historial/compartir`	M	`{ historial_id, profesional_invitado_id }`

No existen `PUT` ni `DELETE`: las evoluciones no se modifican ni se borran.

Archivos

MÉTODO	RUTA	ROLES	NOTAS
POST	`/archivos`	A, R	`multipart/form-data`, campo `archivo`, más `paciente_id` y `turno_id`
GET	`/archivos/:paciente_id`	Todos	Lista de estudios del paciente
GET	`/uploads/:archivo`	Todos	Descarga el archivo; regenera recibos de efectivo faltantes. `404` si no existe

Turnos

MÉTODO	RUTA	ROLES
GET	`/turnos`, `/horarios`	Todos

MÉTODO	RUTA	ROLES
POST	`/turnos`	Todos menos M
PUT	`/turnos/:id`	Todos
DELETE	`/turnos/:id`	Todos
PUT	`/turnos/:id/estado`	Todos menos M
POST	`/turnos/:id/pagar`	Todos menos M
POST	`/turnos/pagar-multiples`	Todos menos M
POST	`/turnos/:id/enviar-email`	Todos menos M

Caja y liquidaciones

MÉTODO	RUTA	ROLES
GET	`/caja/activa`, `/caja/historial`	A, R
POST	`/caja/abrir`, `/caja/cerrar`	A, R
GET	`/liquidaciones?profesional_id=&fecha_desde=&fecha_hasta=`	A, R
GET	`/pagos/profesional/:id`	A, R
POST	`/pagos/profesional/:id`	A, R
GET	`/pagos/paciente/:id`	Todos menos M

Soporte

MÉTODO	RUTA	ROLES
POST	`/api/soporte`	Todos

6 MÓDULOS

Módulos

Cada módulo tiene su pantalla en `frontend/src/components/` y su API en `MedCloud-Backend/src/routes/` y `src/controllers/`. Los permisos de cada endpoint están en api.md.

Sesión y seguridad

Estado: en uso · **Permisos:** todos

Archivos

- Frontend: `LoginPage.jsx`, `ForgotPassword.jsx`, `ResetPassword.jsx`, `ProtectedRoute.jsx`, `App.jsx`, `utils/auth.js`, `utils/useInactivityTimer.js`
- Backend: `controllers/auth.controllers.js`, `controllers/passwordReset.controllers.js`, `middlewares/auth.middleware.js`, `config.js`, `utils/roles.js`

Reglas

- Sesión JWT de 8 h firmada con `JWT_SECRET`; sesión única por usuario; cierre por 15 minutos de inactividad.
- Cambio de contraseña obligatorio en el primer ingreso: el frontend envía el token recibido en el login de forma explícita, porque todavía no está guardado.
- Un `401` cierra la sesión en el navegador y lleva al ingreso.

Usuarios y roles

Estado: en uso · **Permisos:** ADMIN y RECEPCION

Archivos

- Frontend: `UsuariosList.jsx`, `UsuariosForm.jsx`, `RolList.jsx`, `RolForm.jsx`
- Backend: `routes/usuarios.routes.js`, `routes/roles.routes.js`, `controllers/usuarios.controllers.js`, `controllers/roles.controllers.js`

Reglas

- Solo un ADMIN gestiona usuarios ADMIN o el rol ADMIN (evita que recepción se cree un administrador).

Pacientes

Estado: en uso · **Permisos:** listado y ficha para todos; alta, edición y baja para ADMIN y RECEPCION; asistencia y pagos ocultos al MEDICO

Archivos

- Frontend: `PacientesList.jsx`, `PacientesForm.jsx`
- Backend: `routes/pacientes.routes.js`, `controllers/pacientes.controllers.js`, `routes/archivos.routes.js`, `controllers/archivos.controllers.js`

Reglas

- El teléfono se normaliza a `549…` (`utils/phoneFormatter.js`).

- Los estudios se suben a `uploads/` y se ven solo con sesión.

Historia clínica

Estado: en uso · **Permisos:** solo MEDICO

Archivos

- Frontend: `HistorialClinico.jsx`
- Backend: `routes/historial.routes.js`, `controllers/historial.controllers.js`

Reglas

- El médico se toma del token. Las evoluciones no se editan ni se borran (Ley 26.529).

Profesionales

Estado: en uso · **Permisos:** consulta para todos; gestión para ADMIN y RECEPCION

Archivos

- Frontend: `ProfesionalList.jsx`, `ProfesionalForm.jsx`
- Backend: `routes/profesionales.routes.js`, `controllers/profesionales.controllers.js`

Reglas

- La pantalla carga el % de honorario del médico y el backend guarda la retención de la clínica (`100 - honorario`).

Turnos

Estado: en uso · **Permisos:** agenda para todos; agendar, confirmar, cobrar y enviar mails para todos menos MEDICO

Archivos

- Frontend: `TurnosList.jsx`, `TurnosForm.jsx`
- Backend: `routes/turnos.routes.js`, `controllers/turnos.controllers.js`, `utils/emailSender.js`, `utils/scheduler.js`, `utils/reciboHelper.js`

Reglas

- El cobro registra el pago en la caja abierta y confirma el turno. En efectivo se genera un recibo HTML en `uploads/`; con transferencia o MercadoPago se adjunta el comprobante.

Caja diaria y liquidaciones

Estado: en uso · **Permisos:** ADMIN y RECEPCION

Archivos

- Frontend: `CajaControl.jsx`, `CajaHistorial.jsx`, `Liquidaciones.jsx`
- Backend: `routes/caja.routes.js`, `controllers/caja.controllers.js`, `routes/pagos.routes.js`, `controllers/pagos.controllers.js`

Cómo se probó (avance del 2026-10-10)

- Prueba aislada de permisos con las rutas reales, tokens de cada rol y una base inexistente (sin scheduler ni mails): 29 rutas × 4 casos (sin sesión, MEDICO, RECEPCION, ADMIN) dan 401/403/acceso según la matriz de api.md; `PUT` / `DELETE /historial/:id` dan 404; un nombre de archivo con `../` se reduce a un nombre dentro de `uploads/`.
- Bloqueo de escalada con base simulada: 16 casos (crear, editar y borrar usuarios ADMIN y el rol ADMIN siendo RECEPCION o ADMIN) dan el resultado esperado.
- `node --check` de todo el backend, `npm run build` del frontend y lint sin problemas nuevos.
- **No se probó contra la base real** (no hay acceso todavía): falta recorrer en el navegador el ingreso, el cambio de contraseña inicial, la apertura de estudios y recibos y el borrado de pacientes con un usuario de cada rol.

A HISTORIAL

Historial de cambios

2026-10-10 — Seguridad: sesión y permisos en todas las rutas

- **Qué cambió:**
  - Todas las rutas de la API exigen sesión, salvo el login, la recuperación de contraseña y el link público de turnos.
  - Los permisos por rol replican los del frontend.
  - Solo un ADMIN gestiona usuarios o el rol ADMIN.
  - El cambio de contraseña solo permite la propia (o cualquiera si sos ADMIN).
  - La historia clínica toma al médico del token, y se quitaron la edición y el borrado de evoluciones.
  - Los archivos de `uploads/` se sirven solo con sesión y sin poder salir de la carpeta.
  - El secreto de las sesiones pasa al `.env` (`JWT_SECRET`).
  - `jsonwebtoken` queda declarada como dependencia.
  - Se corrigió el borrado de pacientes, que nunca respondía.
  - Se corrigieron los links de recibos que apuntaban a `localhost`.
  - La sesión vencida ahora lleva al ingreso.
- **Módulos:** sesión, usuarios, roles, profesionales, administrativos, pacientes, historia clínica, archivos, turnos, caja, liquidaciones.
- **Backend/API:**
  - Nuevos: `GET /historial/paciente/:id`, `src/config.js`, `src/utils/roles.js`, y `permitirRoles` / `negarRoles` en `auth.middleware.js`.
  - Eliminados: `PUT/DELETE /historial/:id`.
  - `GET /uploads/:archivo` ahora requiere sesión.
  - Falta de token: ahora `401` (antes `403`).
  - `DELETE /pacientes/:id` responde `204`.
- **Frontend:**
  - Nuevo `utils/archivos.js`.
  - Estudios y recibos se abren con sesión desde `PacientesForm`, `PacientesList`, `HistorialClinico`, `Liquidaciones` y `TurnosList`.
  - El cambio de contraseña inicial envía el token.
  - Aviso de "Sesión vencida" en `App.jsx`.
- **Base de datos:** sin cambios (se usan `sp_GetRoles` y `sp_GetUsuarioById` para el control de escalada).
- **Configuración:** nueva variable obligatoria `JWT_SECRET` y nuevo `MedCloud-Backend/.env.example`. Al desplegar se cierran todas las sesiones una vez.
- **Pendiente:** el link público de confirmación/cancelación de turnos (`/turnos/accion-publica`, cambio todavía sin commitear) necesita un token firmado antes de subir a producción.
- **Pruebas:** ver modulos.md.
- **Commits:** `0048fc7` (ignorar archivos del servidor), `bc7dae9` (JWT_SECRET), `a9599f1` (sesión y roles en rutas), `63cf961` (borrar paciente), `88db6d9` (frontend: archivos con sesión y sesión vencida).